Rate Limiting Algorithms: Token Bucket vs Sliding Window

five ways to count, and which one to pick

Watch the 4-min video

> **In one line:** fixed window is cheap but allows double bursts at the boundary, sliding windows are stricter (exact with a log, approximate with two counters), token bucket allows controlled bursts with two numbers per key, and leaky bucket smooths traffic to a steady rate. For a public API, a token bucket per key is the usual choice.

1 The problem: how do you actually count?

Your public API allows **100 requests per minute per API key**, with short bursts of up to **20**. Simple to say. But to enforce it, every request needs a quick yes or no, and that answer depends on how you count. There are five classic algorithms. Each one trades three things differently: **accuracy**, **memory per key** and **burst behaviour**.

2 Fixed window counter

Keep one counter per key, per minute, for example under the key `rl:key42:12:00`. Increment it on each request, reject above 100, and start a new counter at the minute boundary. It is tiny (one integer) and fast (one increment).

But it has a **boundary problem**. The counter only knows which minute a request fell in, not how close together requests were.

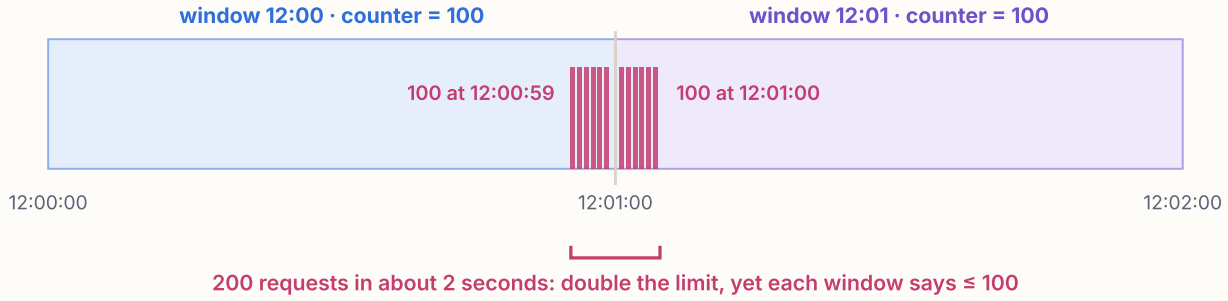


Figure 1. The fixed-window boundary burst. Each window stays within its limit, but 200 requests arrive within about two seconds.

3 Sliding window log and sliding window counter

The **sliding window log** fixes the boundary problem. Store the timestamp of every request in a sorted set. On each request, drop timestamps older than 60 seconds and count what's left. It is **perfectly accurate**: there are never more than 100 requests in any 60 seconds. But memory grows with the limit: up to 100 timestamps per key. With a million keys and higher limits, that is hundreds of millions of entries.

The **sliding window counter** is the practical compromise. Keep just two counters: the current window and the previous one. Then estimate the last 60 seconds by weighting the previous counter by how much of it still overlaps the sliding window:

```
estimate = current + previous × (1 − elapsed / window)
```

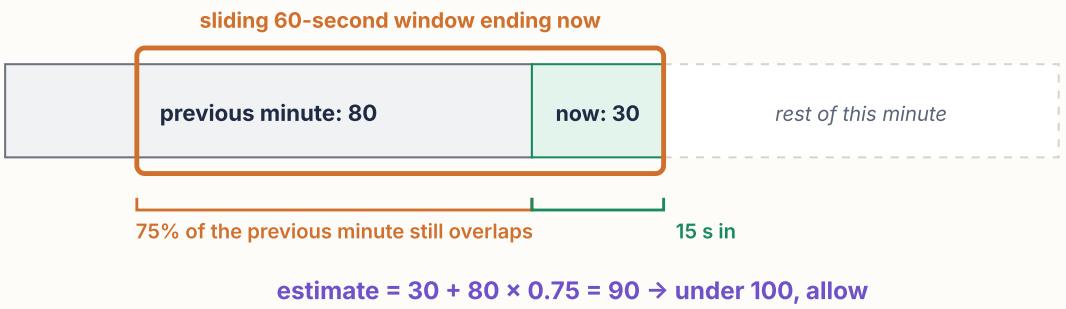


Figure 2. 15 seconds into this minute, 45 of the previous 60 seconds still fall inside the sliding window, so the previous count is weighted by 0.75.

It assumes the previous minute's requests were spread evenly, so it is an approximation, but accurate to within a few percent in practice, with only two counters per key.

4 Token bucket and leaky bucket

The **token bucket** thinks differently. Each key has a bucket that holds up to **20 tokens** and refills at **100 per minute** (one token every 0.6 s). Each request takes one token; with none left, it is rejected with 429. A quiet client builds up a full bucket, so it can burst 20 requests at once, but over time it can never exceed the refill rate. You store only **two numbers**: tokens left and the last refill time. Refill is computed lazily on each request: `tokens = min(20, tokens + elapsed × rate)`, so no background timer is needed.

The **leaky bucket** is the mirror image. Requests join a queue that drains at a constant rate, say one request every 600 ms. If the queue is full, new requests are dropped. It smooths traffic perfectly, which protects a fragile downstream service, but bursts wait in line instead of being served right away.

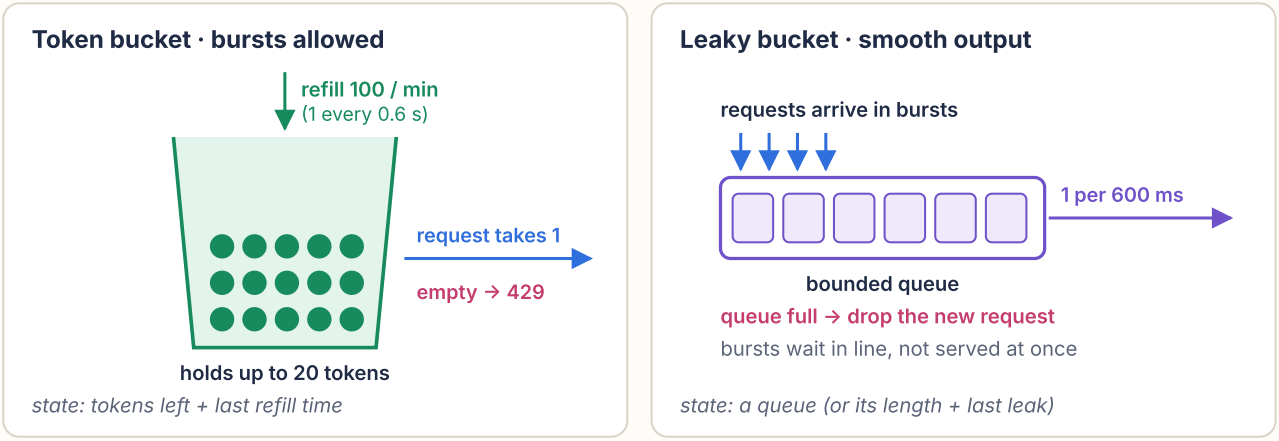


Figure 3. Left: the token bucket stores capacity for bursts. Right: the leaky bucket turns bursty input into a steady output rate.

WORKED EXAMPLE: ONE KEY, TWO ALGORITHMS

A client has been idle, so its bucket is full (20 tokens). It sends 25 requests in one second. The first **20 succeed**, the next 5 get **429**. After that it earns one token every 0.6 s, so about 1.7 requests per second.

Over its first full minute it can make at most **20 + 100 = 120** requests: the stored burst plus the refill. That is the honest summary of a token bucket: *the rate over the long run, plus up to one bucket of extra at any moment*.

With a sliding window counter instead, the same client gets at most 100 in any 60 seconds, all of which it may spend in the first second; there is no separate burst size to tune.

A client that was quiet sends 60 requests at once. What gets through in the first 6 seconds?

Fixed window	up to 60 at once (100 left in the minute), or 200 across a boundary
Sliding window	up to 60 at once, but never more than 100 in any 60 s
Token bucket	20 at once, then 1 every 0.6 s: 30 by t = 6 s, rest 429
Leaky bucket	1 every 0.6 s from a queue: about 10 by t = 6 s, others wait or drop

Figure 4. The same burst under each algorithm (limit 100 per minute, bucket size 20, leak one every 0.6 s).

5 Which one for a public API?

Algorithm	State per key	Accuracy	Bursts	Use when
Fixed window	1 counter	Up to 2× at edges	Double burst at boundary	Approximate is good enough
Sliding log	1 timestamp per request	Exact	None beyond the limit	Low limits, must be exact
Sliding counter	2 counters	Within a few %	None beyond the limit	Strict per-minute limits, cheaply
Token bucket	tokens + timestamp	Exact rate	Up to bucket size	Public APIs (most common)
Leaky bucket	queue	Exact output rate	Queued, not served	Downstream needs a steady rate

The **token bucket** is the usual choice. It allows the friendly bursts that real clients make (a page load firing ten calls at once), it is cheap, and it is easy to explain as “a rate plus a burst”. AWS API Gateway and Stripe both use token buckets. The **sliding window counter** is a great alternative when you want a strict per-minute limit without the fixed window's double burst. Use a **leaky bucket** when a consumer needs a steady rate, and a **fixed window** only when approximate is good enough.

COMMON MISTAKES

- Using a fixed window and quoting the limit as exact: a client can get twice the limit across a boundary.
- Storing a sliding log for high limits: 10,000 requests per minute means 10,000 timestamps per key.
- Running a background job to refill every bucket. Compute the refill lazily from the elapsed time.

6 Key terms

Fixed window: one counter per key per time window

Sliding counter: two weighted counters, approximate

Lazy refill: compute tokens from elapsed time

Boundary burst: 2× the limit across a window edge

Token bucket: refill rate + bucket size (burst)

429: Too Many Requests

Sliding log: exact, one timestamp per request

Leaky bucket: queue drained at a fixed rate

Burst: short spike above the average rate

7 Interview questions

Q1. Compare token bucket and sliding window. BASIC

Token bucket stores tokens and a timestamp, refills at a steady rate and allows bursts up to the bucket size. Sliding window counts requests in the last N seconds, exactly with a log or approximately with two weighted counters: stricter, smoother, no burst allowance.

Q2. Which would you use for a public API, and why? BASIC

A token bucket per API key, for example 100 per minute with a burst of 20. Real clients are bursty, it needs two numbers per key, and “rate plus burst” is easy to document.

Q3. Sliding window counter: previous minute 80 requests, current 30, 15 s in, limit 100. Allow? SCENARIO

Estimate = 30 + 80 × (1 − 15/60) = 30 + 60 = 90. Under 100, so allow.

Q4. Token bucket, capacity 20, refill 100/min. A full bucket gets 25 requests in one second. What happens? SCENARIO

The first 20 succeed and 5 get 429. Then about one token every 0.6 s, so the next request succeeds 0.6 s later.

Q5. A downstream SMS provider accepts at most 10 messages per second. Which algorithm? SCENARIO

A leaky bucket (a queue drained at 10/s). Bursts of messages wait in the queue instead of being rejected or overloading the provider; reject only when the queue is full.

Q6. Memory for 1 million keys at 1,000 requests/minute: sliding log vs token bucket? TRICKY

Sliding log keeps up to 1,000 timestamps per key: up to a billion entries, tens of GB in Redis. Token bucket keeps two numbers per key: about 2 million values, tens of MB. That is why logs are only used for low limits.

Q7. Can a token bucket let through more than the stated per-minute rate? TRICKY

Yes, in a single minute: capacity + rate × time. With 20 + 100, up to 120 in the first minute after a quiet period. Over the long run it averages the refill rate.

Next: SD-C-013 · Distributed Rate Limiting backendroadmap.com